



UNIVERSITATEA TEHNICA

DIN CLUJ-NAPOCA

CENTRUL UNIVERSITAR NORD DIN BAIA MARE

FACULTATEA DE INGINERIE

DEPARTAMENTUL DE INGINERIE ELECTRICĂ, ELECTRONICĂ ȘI CALCULATOARE
PROGRAMUL DE STUDII CALCULATOARE

PLATFORMĂ DE LIVE STREAMING PENTRU JOCURI

PROIECT SISTEME DISTRIBUITE

Student: **Eduard HAIDU**

2024

Cuprins

Capitolul 1	Introducere	1
1.1	Context general	1
1.2	Obiectivele proiectului	1
1.3	Analiza MoSCoW	1
Capitolul 2	Arhitectura Aplicației	3
2.1	Modulul de Ingestie	3
2.2	Modulul de Management și API	3
2.3	Modulul de Procesare Media	4
2.4	Interfața (Frontend)	4
Capitolul 3	Fluxul de lucru	5
Capitolul 4	Concluzii	6

Capitolul 1. Introducere

1.1. Context general

Acest proiect reprezintă o platformă distribuită de live streaming, proiectată pentru a gestiona întregul ciclu de viață al transmisiunilor video, de la ingestie RTMP până la redare HLS în browser. Construită pe un stack modern de Java 21 și Spring Boot, arhitectura utilizează Apache Kafka pentru orchestrarea evenimentelor în timp real și Red5 Server pentru preluarea fluxurilor video (de ex. din OBS). Sistemul include un pipeline automatizat de transcodare video folosind FFmpeg, stocare scalabilă de obiecte prin MinIO și funcționalități complete de VOD (Video on Demand), oferind o experiență similară platformelor consacrate precum Twitch, dar într-o infrastructură proprie, modulară.

1.2. Obiectivele proiectului

În realizarea acestei platforme de live streaming, ne-am propus să atingem următoarele obiective:

- Utilizatorii să poată transmite gameplay-ul în timp real folosind OBS sau alt program asemănător
- Alți utilizatori să se poată conecta și să vizualizeze în timp real gameplay-ul transmis
- Înregistrările live să fie stocate în momentul încheierii transmisiei în direct
- Utilizatorii să poată viziona reluările transmisiilor.
- Utilizatorii să poată crea conturi și asocierea transmisiilor live cu utilizatorul corespunzător.

1.3. Analiza MoSCoW

Pentru a determina care cerințe sunt cele mai importante în cadrul aplicației și pentru a le prioritiza mai eficient, am efectuat o analiză MoSCoW pentru a avea o structură clară care ne ajută să categorizăm cerințele în funcție de importanța lor.

Categorie	Prioritate	Funcționalități din Proiect
Must Have	Critice pentru funcționarea MVP (Minimum Viable Product).	• Ingestie RTMP (via Red5 Server) • Transcodare HLS în timp real (FFmpeg) • Redare Live Stream (Video.js) • Autentificare securizată cu Stream Key • Stocare obiecte (MinIO) pentru segmente video
Should Have	Importante, dar sistemul ar putea funcționa (limitat) fără ele.	• Înregistrare automată VOD (Video on Demand) • Pagini de profil utilizator (Canale) • Arhitectură bazată pe evenimente (Kafka) pentru scalabilitate • Autentificare utilizatori via JWT
Could Have	Dorite, menționate ca optimizări sau pași următori.	• Integrare CDN pentru distribuție globală (menționat la Performance Tuning) • Transcodare Multi-bitrate (Adaptive Bitrate Streaming - ABR)* • Chat Live pentru interacțiune (specific platformelor Twitch-like) • Ștergerea automată a VOD-urilor vechi (Lifecycle policies)
Won't Have	Excluse din iterația curentă.	• Protocol WebRTC (latență sub-secundă) – se folosește strict HLS • Aplicații native de mobil (Frontend-ul este HTML/JS static) • Sistem de monetizare/abonamente • Analytics avansat (doar viewer count este implementat)

Tabela 1.1: Analiza MoSCoW a cerințelor pentru Platforma de Streaming Java

Capitolul 2. Arhitectura Aplicației

Arhitectura aplicației este una distribuită și **Orientată pe Evenimente (Event Driven Architecture)**. Aceasta este proiectată pentru a decupla ingestia fluxului video de procesarea acestuia, asigurând scalabilitate și performanță.

Aplicația este construită folosind tehnologii care au la bază limbajul Java, și este compusă din următoarele module principale:

- **Modulul de ingestie**
- **Modulul de procesare media**
- **Modulul de management și API**
- **Modulul de interfață (Front-End)**

Iar pentru a lega aceste module, arhitectura se bazează pe trei piloni de infrastructură:

- **Apache Kafka:** Coloana vertebrală a sistemului. Asigură comunicarea asincronă între Red5, Backend și Transcoder. Dacă modulul de transcodare cade, mesajele rămân în Kafka, asigurând reziliența.
- **MinIO:** Este utilizat pentru stocarea efectivă a fișierelor video. Este folosit atât pentru fragmentele efemere de Live, cât și pentru stocarea pe termen lung a VOD-urilor.
- **PostgreSQL:** Baza de date relațională care păstrează structura utilizatorilor, relațiile dintre canale și istoricul transmisiunilor.

2.1. Modulul de Ingestie

Modulul de ingestie este implementat cu ajutorul tehnologiei **Red5 RTMP Server**. Acesta reprezintă punctul de plecare al sistemului pentru streameri.

- **Rol:** Funcționează ca un server RTMP dedicat care acceptă conexiuni de la software-uri de broadcasting precum OBS Studio.
- **Metoda de funcționare:**
 - Nu procesează video-ul final, ci doar preia fluxul brut transmis de programul de broadcasting
 - Validează cheile de streaming (Stream Keys) printr-un apel REST către Backend.
 - Publică evenimente de ciclu de viață (START/STOP stream) în topicul Kafka **live-stream** pentru a notifica restul sistemului.

2.2. Modulul de Management și API

Acesta este "creierul" aplicației, gestionând logica de business și persistența metadatelor.

- **Tehnologie:** Acest modul a fost implementat cu ajutorul framework-ului Spring Boot, bazat pe Java.
- **Modul de funcționare:**
 - Expune un API de tip REST pentru înregistrare, autentificare (securizată folosind JWT) și gestionarea canalelor.

- Consumă evenimente din Kafka pentru a actualiza starea stream-urilor în baza de date (ex: marchează un stream ca fiind LIVE).
- Interacționează cu baza de date **PostgreSQL** pentru stocarea utilizatorilor și a metadatelor stream-urilor.

2.3. Modulul de Procesare Media

Acesta este componenta care efectuează munca intensivă (CPU-intensive), fiind separată de API pentru a nu bloca cererile utilizatorilor.

- **Rol:** Transformă fluxul RTMP brut în format HLS (HTTP Live Streaming) compatibil cu browserele web.
- **Tehnologie:** Modulul este implementat prin intermediul unui Wrapper Java dezvoltat peste tehnologia **FFmpeg**
- **Modul de funcționare**
 - Ascultă evenimentele de START din Kafka.
 - Pornește procesul de transcodare (H.264 video, AAC audio).
 - Încarcă segmentele ".ts" și playlist-urile ".m3u8" în timp real în **MinIO** (Object Storage).
 - La finalul transmisiei, se ocupă de arhivarea stream-ului pentru VOD (Video on Demand).

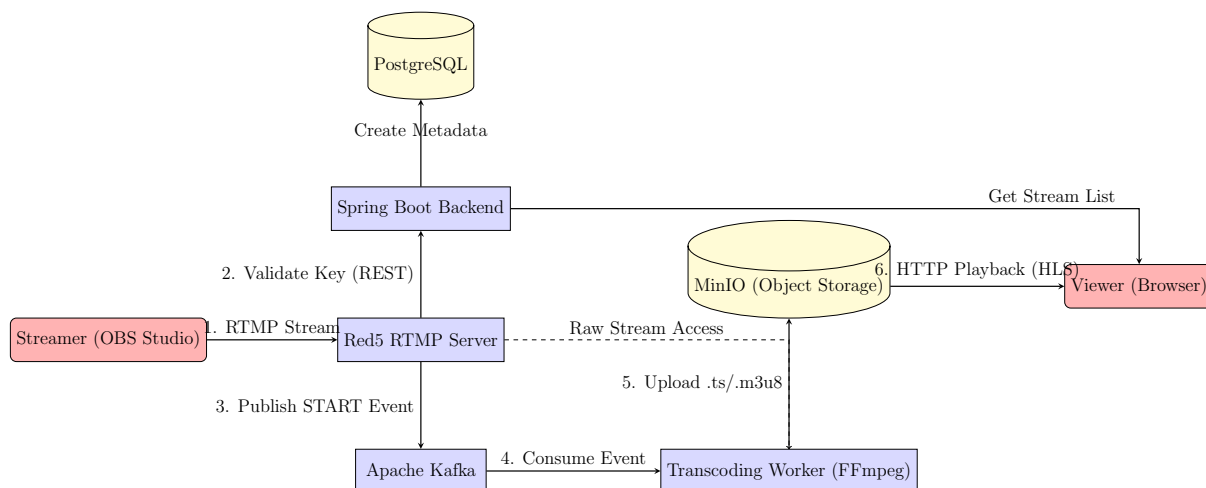
2.4. Interfața (Frontend)

Aplicația oferă o interfață intuitivă pentru utilizator, decuplată complet de logica de backend.

- **Tehnologie:** Intefăța este implementată folosind ThymeLeaf cu HTML și JavaScript static, fără framework-uri complexe, pentru performanță maximă.
- **Componente:** Utilizează **Video.JS** pentru a reda fluxurile HLS direct din MinIO (prin proxy-ul backend-ului)
- **Modul de funcționare:** Oferă pagini pentru vizualizarea stream-urilor live, navigarea în arhiva VOD și setările contului de streamer.

Acest design modular permite scalarea independentă a componentelor, în funcție de nevoile aplicației care apar pe parcurs.

Capitolul 3. Fluxul de lucru



Descrierea pașilor

1. **Ingestie:** OBS se conectează la Red5 folosind Stream Key.
2. **Validare:** Red5 verifică cheia prin API-ul Spring Boot. Dacă e validă, se creează metadata în PostgreSQL.
3. **Notificare:** Red5 publică evenimentul **START** în topicul Kafka.
4. **Transcodare:** Worker-ul consumă evenimentul, pornește FFmpeg și convertește fluxul în HLS.
5. **Distribuție:** Segmentele video sunt încărcate în timp real în MinIO.
6. **Redare:** Utilizatorul accesează interfața web, primește lista stream-urilor din Backend și încarcă video-ul din MinIO.
7. **Oprire:** În momentul încheierii transmisiei live, Red5 publică evenimentul **STOP** în topicul Kafka.
8. **Arhivare:** Workerul consumă evenimentul **STOP** și începe arhivarea videoclipului într-un folder dedicat în MinIO. Apoi, segmentele de stream sunt curățate, iar videoclipul va putea fi vizionat din zona "Past Broadcasts" a canalului.

Capitolul 4. Concluzii

În concluzie, proiectul validează viabilitatea unei platforme de streaming *self-hosted* construite pe tehnologii open-source moderne. Prin combinarea puterii **Java** cu flexibilitatea **Kafka** și **FFmpeg**, s-a obținut un sistem robust, capabil să gestioneze latența și debitele mari de date specifice streaming-ului video, oferind în același timp o experiență de utilizare fluidă, similară platformelor comerciale consacrate.

Repository GitHub